



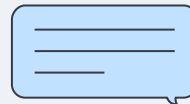
Natural Language processing

Group 17

Adriano Machado (up202105352)

Félix Martins (up202108837)

Francisco da Ana (up202108762)

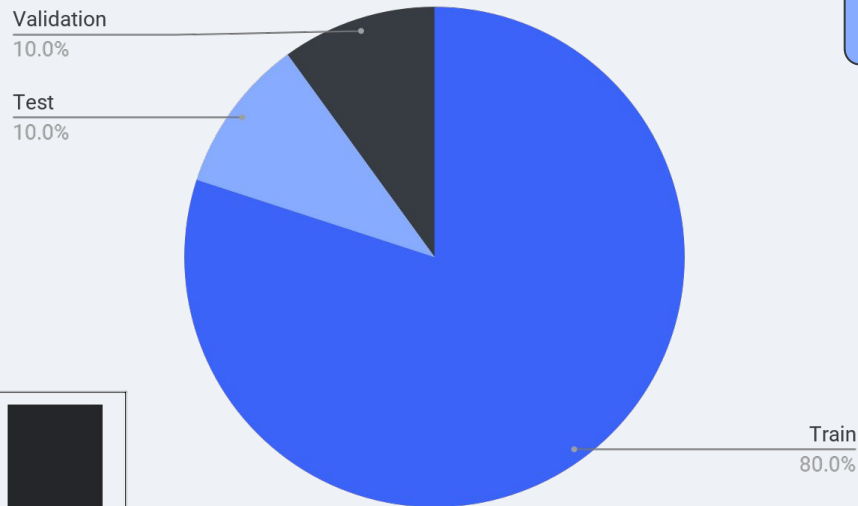
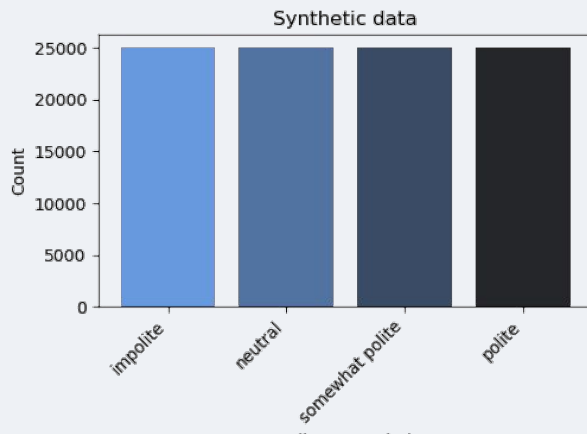


Polite Guard Dataset

Goal: Classification of text into **polite**, **somewhat polite**, **neutral** or **impolite**

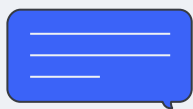
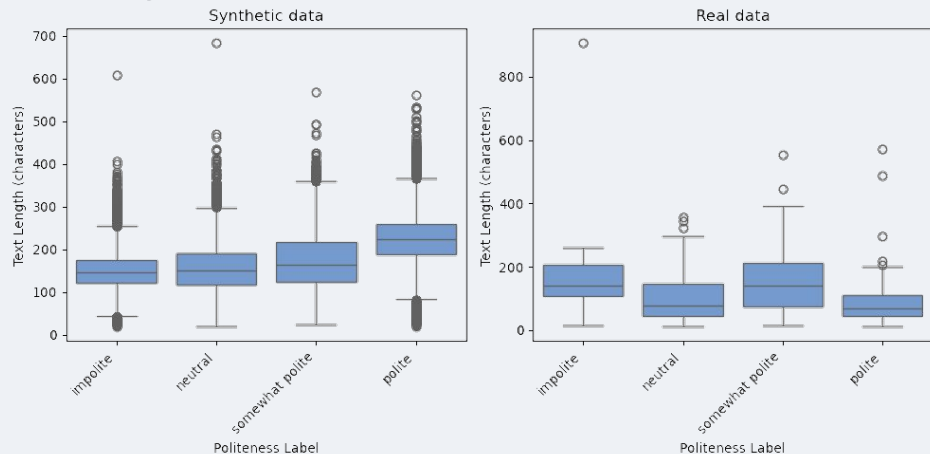
Synthetic dataset: 100k samples generated by LLMs, **200 real** annotated examples with personal identifiers removed

Key Considerations: No personal identifiers, even distribution of classes (25% of each class in every split)



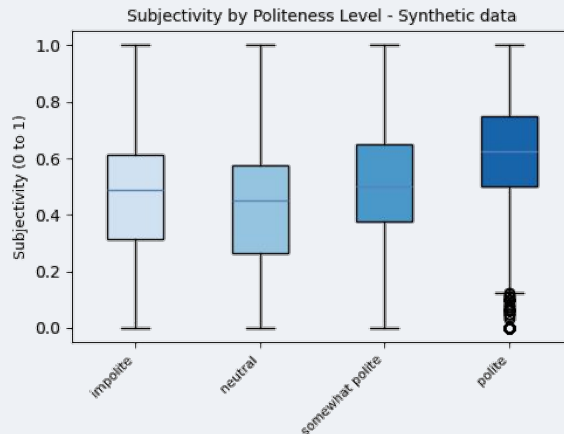
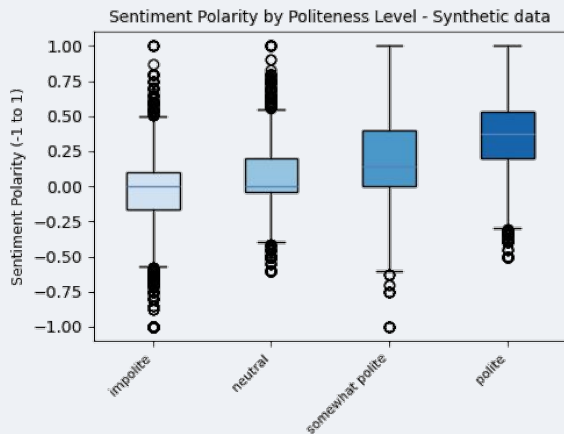
Exploratory Data Analysis

- **No missing or duplicated** values
- The synthetic datasets exhibit a tendency where **longer texts** are associated with **higher politeness** levels. This pattern does not exist in the real data.
- Analyzing the **word clouds**, we notice **Impolite** language conveys frustration through words like "seriously" or "complaining" while **Polite language** is marked by expressions of **helpfulness** and respect, including words like "happy" and "assistance"



Exploratory Data Analysis

- We also performed sentiment analysis on our dataset
- **Sentiment Polarity** is directly related to the politeness level.
- In terms of **Subjectivity** the relationship is less clear, but **polite** texts tend to be **more subjective** than the **impolite** ones



Pre-Processing

Goal: Clean and normalize raw text data, remove noise

Pre-Processing Steps (varies by feature representation):

- Remove non-alphanumeric characters
- Convert to lowercase
- Remove stopwords (for BoW-unigrams and TF-IDF)
- Lemmatization, with POS tagging for accuracy

Impact on Vocabulary Size: Before pre-processing we had **10,943 tokens**. After pre-processing, including stopwords we have **6,932 tokens**, and without stopwords we have **6,811 tokens**.

Your flight has been rescheduled for 10:00 AM tomorrow. Please check the airport's website for any updates or changes

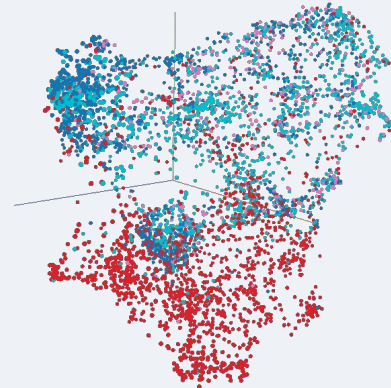
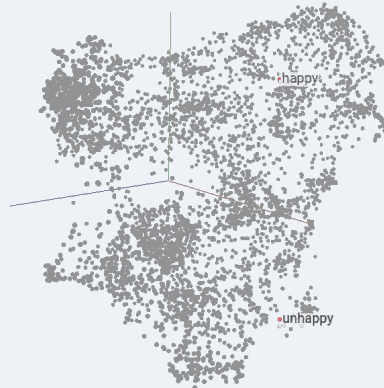
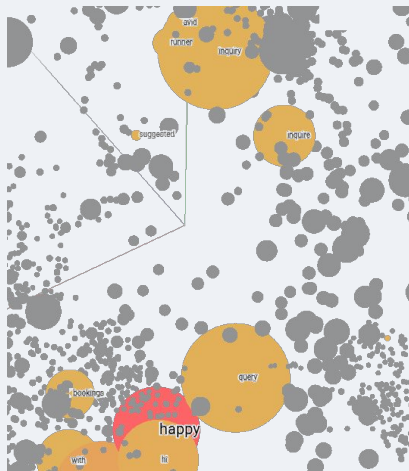
**After
pre-processing**

flight reschedule 10 00 tomorrow please check airport website update change

Feature Representation Techniques

- We explored the **interpretability** of our **embeddings** by visualizing them in 3D via TensorBoard, reducing dimensionality with **UMAP**.
- The resulting plot shows significant structure. Notably, there's a **clear separation based on politeness**. Words like **'happy'** (near 'query,' 'avid,' 'inquire,' 'suggested') appear distant from words like **'unhappy'** (near 'complain,' 'irresponsible,' 'forget,' 'wrong,' 'frankly'). This confirms the **embeddings learned semantic nuances** relevant to politeness

Method	Key Feature	Output Dimensions
Bag of Words	Unigrams/Bigrams	Vocabulary
TF-IDF	Term Weighting	Vocabulary
Word2Vec	Semantic Embeddings	200
SVD	Sparse Reduction	200



Results

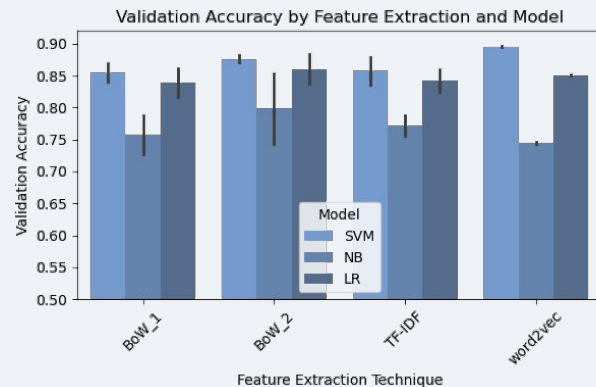
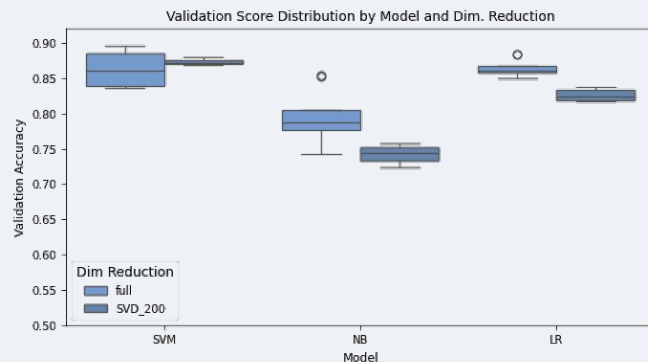
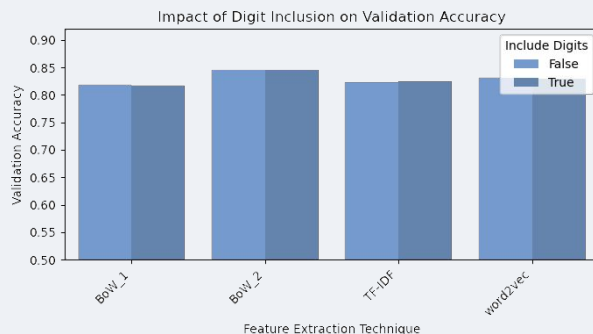
Models: Naive Bayes, Logistic Regression and SVM

Hyperparameter Tuning: Different parameters tested for each model

Logistic Regression and **SVM** are the **most effective** models for this task, consistently outperforming Naive Bayes

While using the full feature set yields slightly higher accuracy, dimensionality reduction via SVD offers comparable performance, especially for **SVM**

Including or excluding digits during preprocessing **does not** affect model performance for the methods tested



Results

Overall Performance

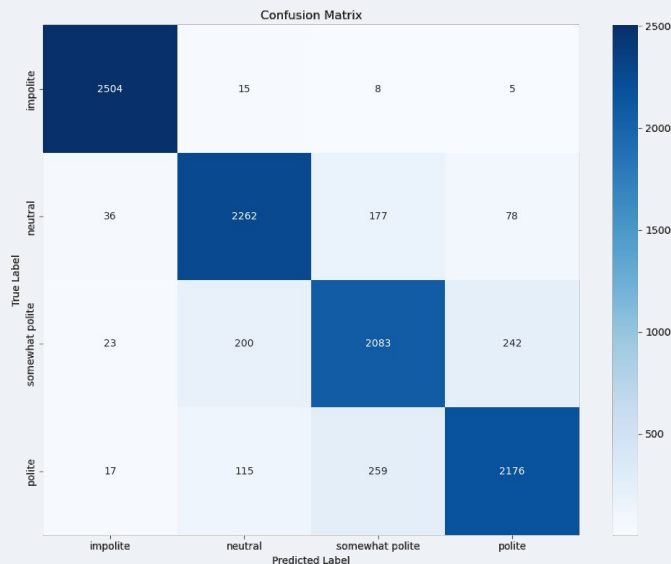
- SVM + Word2Vec achieved **88.48% accuracy** and balanced performance across politeness levels.
- Strong robustness with consistent macro & weighted F1 scores (~88.5%).

Class-Specific Analysis

- **Impolite: Best performance** (F1 = 97.97%), very high precision & recall.
- **Somewhat Polite: Most misclassifications**, often confused with neutral & polite.

Error Patterns & Key Takeaways

- Misclassifications occur mostly between adjacent politeness levels → suggests a natural spectrum.
- Even humans would struggle to classify highly polite samples → subjectivity in politeness perception.
- Real-world use case: effective for detecting offensive & harmful comments due to high impolite-class accuracy.



Error Analysis

For our best model, we can try to understand what is leading it to make mistakes by analysing concrete examples: for example, misclassification as polite instead of impolite.

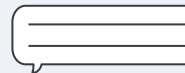
"Are you **really trying** to return a toy that's been opened and played with? Our return policy is clear - items must be in their original packaging with all tags attached. If you'd like to make a purchase, I'd be **happy to help** you find something else."

Word2Vec embeddings may overemphasize "happy to help" and "find something else," diluting the harshness of "really trying."

"While I **understand** that the progress tracking system can be frustrating, my job is to provide assistance, not to **deal with your attitude**. If you have specific questions or issues, I'm **here to help**."

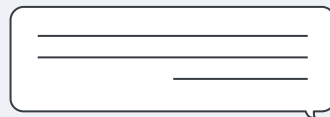
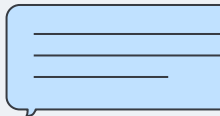
Word2Vec might weight "understand" and "help" highly, missing the negative "attitude" context.

Our model struggles with understanding full sentence context, which leads to failure in cases with mixed contents.



Conclusions

- Simple NLP approaches reached 88% F1, competitive in comparison to state-of-the-art's 92%.
- Basic feature extraction techniques (BoW, TF-IDF, Word2Vec) fail to understand the full context of text.
- This results in misclassifications in contents with mixed tones.
- Excelling at **impolite** text detection suggests the model is a valuable tool for a real world scenario of offensive content filtering.



Extra: Experiments for Validation Process

In order to optimize the predictive performance of our models, we had to find the configuration (*Feature Representation, Model*) that maximized the predictive performance in the validation set.

Feature Configuration Testing

Tested several feature extraction configurations combining:

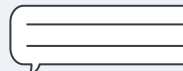
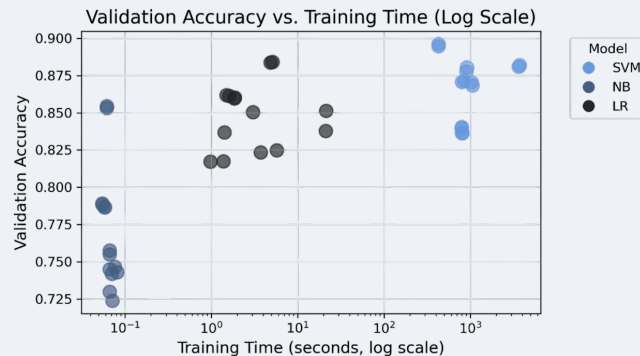
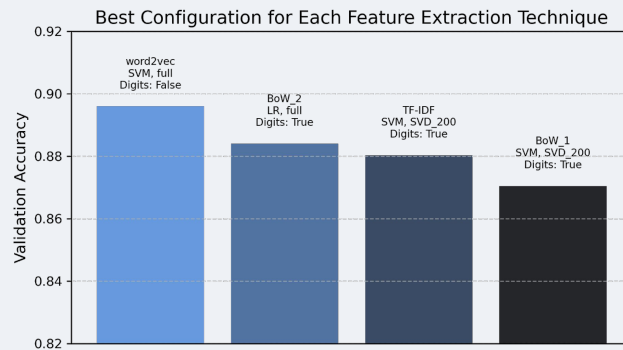
- 4 representation methods: BoW unigram, BoW bigram, TF-IDF, Word Embeddings
- Dimensionality reduction options (for sparse matrices)
- Digit inclusion/exclusion in preprocessing
- 3 possible models: NB, LR and SVM
- In total: $(2 \times (3 \times 2 + 1)) \times 3 = 42$ possible combinations

Model Optimization

- Performed hyperparameter tuning for each feature configuration
- Each configuration tested with dozens of model parameters
- Total: Hundreds of model tests across all feature configurations

Key Results

- Best performer: **Word Embeddings + SVM (no digits)**
- Runner-up: **BoW bigram + Logistic Regression (with digits, no dimensionality reduction)**



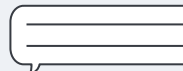
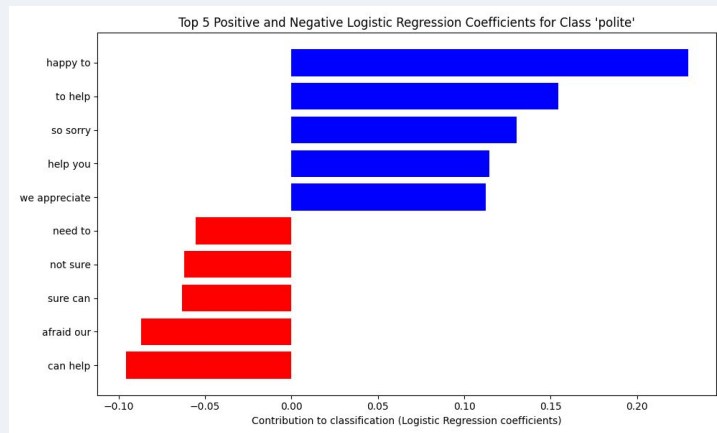
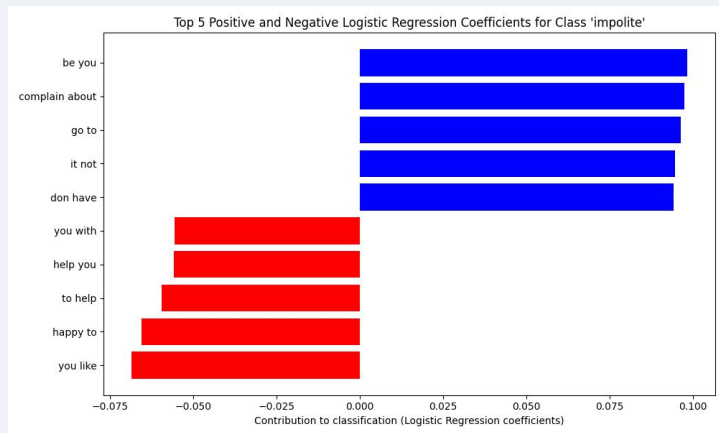
Extra: Feature Importance Results

To interpret the predictions of our best models, we analyzed feature importances:

- SVM + Word2Vec: Embeddings lack direct interpretable features
- **Logistic Regression + BoW bigrams:** offers clear insights based on bigrams

We plotted Logistic Regression coefficients to reveal bigram impacts on predictions.

For each class, plots show positive values and negative values, corresponding to the contributions towards predicting that class:



References

- Polite Guard Dataset
- Polite Guard state-of-the-art model
- Other NLP projects and sources:
 - https://github.com/kk7nc/Text_Classification?tab=readme-ov-file
 - https://github.com/lpq29743/text_classification
 - <https://github.com/Christophe-pere/Text-classification/tree/master>
 - https://edumunozsala.github.io/BlogEms/jupyter/nlp/classification/embeddings/python/2020/08/15/Intro_NLP_WordEmbeddings_Classification.html

